

Riassunto: Concorrenza e Executor Framework (Java)

L'obiettivo di questo modulo è imparare a gestire il multithreading in Java in modo professionale, efficiente e sicuro, abbandonando la creazione manuale dei Thread.

1. Il Problema: Tasks vs Threads

Nelle applicazioni (es. un Web Server), il lavoro viene diviso in **Task** (unità di lavoro discrete e indipendenti, come "gestire la richiesta di un singolo client").

I Task vengono eseguiti dai **Thread** (il meccanismo fisico del sistema operativo).

Come associamo i task ai thread?

1. **Esecuzione Sequenziale (Single Thread):** Il server gestisce un client alla volta. Pessimo tempo di risposta, la CPU si gira i pollici.
2. **Un Thread per ogni Task (`new Thread(task).start()`):** Apparentemente risolve il problema, ma è **pericolosissimo**. Se arrivano 10.000 client, crei 10.000 thread. Creare e distruggere thread consuma enormi risorse (memoria RAM) e porta rapidamente all'instabilità e al crash della JVM (*OutOfMemoryError*).

2. La Soluzione: L'Executor Framework

L'Executor Framework **separa la definizione del Task dal meccanismo che lo eseguirà**.

Invece di dire "Crea un nuovo thread e fagli fare questo", tu dici all'Executor "Ecco il task, pensaci tu a eseguirlo seguendo la tua politica".

L'interfaccia primaria è banalissima:

```
public interface Executor {  
    void execute(Runnable command);  
}
```

Le Politiche di Esecuzione (Execution Policies)

Disaccoppiare il task dall'esecuzione permette di definire regole precise:

- In quale thread verrà eseguito il task?
- In che ordine? (FIFO, LIFO, Priorità).
- Quanti task possono girare contemporaneamente?
- Se il sistema è sovraccarico, quale task rifiutiamo e come avvisiamo l'app?

3. I Thread Pool (Fabbrica Executors)

Invece di creare thread infiniti, si crea una "Piscina di Thread" (Thread Pool). I thread nascono, eseguono un task preso da una coda (Queue), e quando finiscono **non muoiono**, ma tornano nella piscina pronti per un nuovo task.

La classe utility Executors fornisce dei *Factory Method* per creare i pool più comuni:

- **newFixedThreadPool(int n):** Crea un pool con un numero *fisso* di thread (es. 10). Se arrivano più di 10 task, vengono messi in coda finché un thread non si libera.
- **newCachedThreadPool():** Ideale per molti task brevi. Non ha limiti di grandezza. Se serve crea un nuovo thread, ma se un thread rimane inattivo per 60 secondi viene ucciso per liberare RAM.
- **newSingleThreadExecutor():** Ha un solo thread. Garantisce che i task vengano eseguiti in modo rigorosamente sequenziale (FIFO).
- **newScheduledThreadPool():** Sostituisce il vecchio java.util.Timer. Permette di eseguire task con un ritardo (delay) o in modo periodico. Internamente usa una DelayQueue.

4. Oltre il Runnable: Callable e Future

Il classico Runnable ha un difetto enorme: il suo metodo public void run() non può restituire alcun risultato (void) e non può lanciare eccezioni controllate. Se il tuo task fa un calcolo complesso, come fai a riprenderti il risultato?

La soluzione di Java 5:

1. **Callable<V>:** È un task che fa un calcolo e restituisce un risultato di tipo V. Ha il metodo V call() throws Exception.
2. **Future<V>:** Quando invii un Callable a un Executor tramite il metodo submit(), l'Executor ti restituisce istantaneamente un oggetto Future. Il Future rappresenta la "promessa" di un risultato che arriverà in futuro.

Metodi vitali del Future:

- get(): **ATTENZIONE, È BLOCCANTE!** Se il risultato è pronto te lo dà, altrimenti blocca il thread chiamante finché il calcolo non termina.
- isDone(): Restituisce true se il task ha finito.
- cancel(): Tenta di interrompere/cancellare il task.

5. ExecutorCompletionService (Il tocco di classe)

Il Problema: Se invii 10 Callable a un Executor, metti i 10 Future in una List, e poi fai un ciclo for chiamando get() su ognuno, sei costretto ad **aspettare i risultati nell'ordine esatto in cui li hai inviati!** Se il task 1 ci mette un'ora e il task 2 ci mette un secondo, tu aspetterai un'ora per leggere il task 1 prima di poter vedere il task 2 (che era già pronto).

La Soluzione: L'ExecutorCompletionService. È un *Decorator* che avvolge un Executor e

contiene al suo interno una `BlockingQueue`.

Invi i task con `submit()`. Quando vuoi i risultati, chiami il metodo **`take()`**. Questo metodo ti restituisce *il primo Future che ha terminato l'esecuzione*, indipendentemente dall'ordine di invio!

POSSIBILI DOMANDE ALL'ESAME (Con risposte per il 30L)

Domanda 1: "Perché è sconsigliato creare manualmente i Thread per ogni richiesta in un server (modello Thread-Per-Task)?"

Risposta da Lode: "Perché porta all'esaurimento delle risorse hardware. Creare e distruggere oggetti Thread ha un costo computazionale elevato. Inoltre, se arrivano picchi di richieste, il sistema istanzia migliaia di thread contemporaneamente saturando la memoria (rischio di `OutOfMemory`) e sprecando cicli di CPU per il context-switching continuo. L'Executor Framework con i Thread Pool risolve questo problema riciclando un numero controllato di thread."

Domanda 2: "Qual è la differenza fondamentale tra un Runnable e un Callable? E a cosa serve un Future?"

Risposta da Lode: "Un Runnable modella un task asincrono che non restituisce alcun valore (il metodo `run` è void) e non può sollevare eccezioni checked. Un `Callable<V>` risolve questo limite: il suo metodo `call()` restituisce un oggetto di tipo V e può lanciare eccezioni. Quando sottopettiamo un Callable a un Executor tramite `submit()`, otteniamo in cambio un `Future<V>`, che è un 'biglietto' che rappresenta il calcolo pendente. Chiamando `future.get()`, il nostro thread si metterà in attesa bloccante finché il risultato asincrono non sarà pronto."

Domanda 3: "Mi spieghi a cosa serve l'ExecutorCompletionService e che vantaggio dà rispetto a ciclare una lista di Future?"

Risposta da Lode: "Se salvo i Future restituiti dall'Executor in una normale List e poi faccio un ciclo iterando con la `get()`, mi vincolo ad aspettare i risultati nello stesso ordine in cui ho sottoposto i task. Se il primo task è molto lento, bloccherò l'elaborazione dei successivi anche se hanno già finito. L'ExecutorCompletionService unisce un Executor a una `BlockingQueue`: man mano che i task terminano, i loro Future vengono spinti nella coda. Chiamando il metodo `take()`, prelevo il primo task *effettivamente completato*, ottimizzando l'asincronia reale."

Domanda 4: "Quali tipi di Thread Pool mette a disposizione la classe factory Executors?"

Risposta da Lode: "I tre principali sono:

1. `newFixedThreadPool(n)`: pool con dimensione massima fissa, ideale per limitare i carichi ed evitare saturazioni (i task extra attendono in coda).
2. `newCachedThreadPool()`: pool elastico senza limiti massimi. Crea nuovi thread se serve, ma uccide automaticamente quelli inattivi da oltre 60 secondi. Ideale per molti task

asincroni di breve durata.

3. `newSingleThreadExecutor()`: pool con un solo thread worker. Garantisce un'esecuzione strettamente sequenziale dei task sottomessi."